AI Fitness Form Corrector

System Analysis & Design
Documentation

Table of Contents

1. Introduction and System Overview 3
2. Problem Statement and Objectives 3
3. Scope, Assumptions and Constraints 3
4. Stakeholders and Actors 4
5. Use Case Descriptions 4
6. Functional Requirements 5
7. Non-Functional Requirements 5
8. Software Architecture 6
9. Data Flow and System Behavior 7
10. DFD Level 1 – Detailed Processing 7
11. Sequence Diagram – Live Workout 8
12. Activity Diagram – Workout Workflow 9
13. State Diagram – Repetition Counter 9
14. Class Diagram – Logical Design 10
15. UI/UX Design and Prototyping 11
16. UI/UX Guidelines 11
17. Component Diagram 11
18. Deployment Diagram and Strategy 12
19. Technology Stack 13
20. API and Internal Interface Documentation 13
21. Testing and Validation Plan 14
22. Model Evaluation and Acceptance Criteria 15
23. Risks and Mitigation 15
24. Privacy, Safety and Limitations 15
25. Future Enhancements 16
26. Conclusion 16

1. Introduction and System Overview

The AI Fitness Form Corrector is a camera-based application designed to help users understand the quality of selected exercises. It receives live video, estimates body pose, derives movement features, evaluates exercise form, counts repetitions, and presents feedback through a Streamlit interface.

The MVP supports Squat, Push-up, and Biceps Curl. A user selects an exercise and video input method, starts a workout, views live metrics, stops the session, and exports a session report.

The system uses a modular Python application. Streamlit provides the interface; OpenCV processes frames; MediaPipe estimates pose landmarks; feature extraction calculates angles and other features; a trained ML model classifies form; a state machine counts repetitions; and a feedback module prepares user-facing guidance.

No login or persistent database is required in the MVP. Session data is held temporarily and can be exported. An LLM assistant is a future enhancement, not an MVP feature.

- Provide near-real-time analysis of supported exercises.
- Estimate body landmarks and exercise-relevant joint angles.
- Classify form using a validated ML model.
- Count repetitions using exercise-specific movement states.
- Display live repetition count, form status, and corrective feedback.
- Export a workout summary without account creation.

2. Problem Statement and Objectives

People often exercise without continuous supervision. Incorrect posture, incomplete range of motion, or inconsistent movement may reduce exercise effectiveness and increase the likelihood of discomfort. One-to-one trainer supervision may not always be available.

This project aims to provide a camera-based assistant that analyzes selected exercises and offers immediate, understandable feedback. It is an educational fitness-support tool, not a medical diagnostic system and not a replacement for professional coaching.

- Detect body landmarks from camera frames.
- Extract joint angles, relative positions, and movement features.
- Classify form as correct or incorrect, subject to confidence and model limitations.
- Count completed repetitions using a state-based movement logic.
- Display live metrics and actionable feedback.
- Generate a downloadable session summary.

3. Scope, Assumptions and Constraints

In scope: browser-based Streamlit camera input and OpenCV device-camera capture; three MVP exercises; frame preprocessing; pose estimation; feature extraction; form classification; repetition counting; feedback; and report export.

Out of scope: registration, authentication, persistent workout history, relational database, mobile application, multi-user trainer portal, and LLM conversational assistant. These are future enhancements.

Assumptions include camera permission, sufficient lighting, a useful viewing angle, and the user being visible in the frame. Pose quality may decrease due to occlusion, poor lighting, motion blur, or unsuitable camera placement.

Model performance depends on dataset quality, labels, subject diversity, and similarity between training and live conditions. The final classifier and thresholds must be selected through validation.

- Both video acquisition options are included in the design: browser camera input and OpenCV device-camera input.
- Only session information required for live feedback and report generation is held temporarily.
- Raw video is not persistently stored by the MVP.
- The report is generated from the in-memory session summary.
- The system must indicate insufficient pose quality rather than present uncertain feedback as reliable.

4. Stakeholders and Actors

The primary external actor is the User. The camera/video source is an external input source. Pose estimation and ML classification are internal software components, not independent human actors.

Actor	Role / Interest
User	Selects exercise and video source, starts/stops workout, views feedback, exports report.
Camera / Video Source	Supplies live frames through browser capture or OpenCV.
Project Team / Maintainers	Prepare and validate models, maintain modules, and deploy updates.

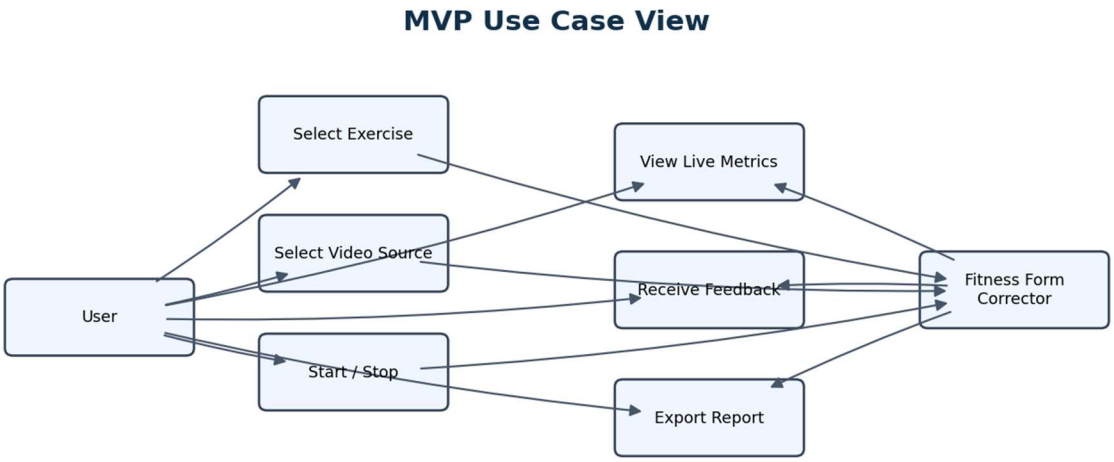


Figure 1. MVP use-case view.

5. Use Case Descriptions

Use Case	Actor	Preconditions	Main Flow	Postcondition
UC-01 Select Exercise	User	Application open.	Choose Squat, Push-up, or Biceps Curl.	Exercise loaded.

Use Case	Actor	Preconditions	Main Flow	Postcondition
UC-02 Select Video Source	User	Camera permission/device available.	Choose browser camera or OpenCV capture; initialize source.	Source ready or clear error shown.
UC-03 Start Workout	User	Exercise and source selected.	Start frame acquisition and initialize analysis state.	Workout active.
UC-04 Analyze Movement	System	Workout active.	Preprocess frame; detect pose; extract features; classify form; update rep state.	Live metrics updated.
UC-05 View Feedback	User	Analysis result available.	View count, form status, corrective feedback.	Current guidance displayed.
UC-06 Stop Workout	User	Workout active.	Stop session and release camera resource.	Session summary finalized.
UC-07 Export Report	User	Summary exists.	Generate downloadable report file.	Report available.

6. Functional Requirements

ID	Requirement	Priority
FR-01	Display Squat, Push-up, and Biceps Curl as supported MVP exercises.	Must
FR-02	Allow browser-based camera or OpenCV device-camera input.	Must
FR-03	Handle camera permission denial or unavailable devices with a clear message.	Must
FR-04	Acquire frames while a workout is active.	Must
FR-05	Preprocess frames as required before pose estimation.	Must
FR-06	Estimate body landmarks using the selected pose model.	Must
FR-07	Derive exercise-relevant features such as joint angles and landmark relationships.	Must
FR-08	Evaluate form using the validated classifier and configured decision logic.	Must
FR-09	Count repetitions using exercise-specific movement states and thresholds.	Must
FR-10	Display live repetition count, form status, and feedback.	Must
FR-11	Avoid confident feedback when landmarks are missing or below quality threshold.	Must
FR-12	Stop the workout and release the camera resource.	Must
FR-13	Maintain temporary session metrics in application memory.	Must
FR-14	Generate a downloadable session report from the session summary.	Must
FR-15	Display pose overlay when supported by the selected implementation.	Should

7. Non-Functional Requirements

ID	Category	Requirement / Acceptance Consideration
NFR-01	Performance	Live feedback should be responsive; measure FPS and end-to-end latency on target hardware.
NFR-02	Accuracy	Evaluate classification and repetition counting on held-out data; report metrics.

ID	Category	Requirement / Acceptance Consideration
NFR-03	Usability	Main workflow should be understandable without account creation.
NFR-04	Reliability	Handle camera, frame-read, and model errors without uncontrolled crashes.
NFR-05	Privacy	No persistent raw video or account data is required by the MVP.
NFR-06	Portability	Run in a documented Python environment and support Docker packaging.
NFR-07	Maintainability	Keep clear module boundaries for video, pose, features, ML, counting, feedback, and reports.
NFR-08	Scalability	Permit future resource upgrades; multi-user scaling is not an MVP requirement.
NFR-09	Accessibility	Use readable text; do not communicate correctness by color alone.
NFR-10	Safety	Feedback is exercise assistance, not medical advice or a guarantee of injury prevention.

8. Software Architecture

The proposed style is Modular Monolithic Architecture. Modules are separated by responsibility but run as one application deployment unit. This reduces operational complexity while supporting testing and extension.

Streamlit is the presentation layer and entry point. A session controller coordinates exercise selection, video source, session state, and result updates. Video input adapters abstract the two acquisition paths. OpenCV handles frame operations; MediaPipe estimates pose; feature extraction calculates movement features; the classifier predicts form; the repetition counter tracks movement phases; the feedback engine combines outputs; and the report generator exports the summary.

A separate REST backend is not assumed. Internal Python interfaces are sufficient for the MVP. Persistent database storage is not included.

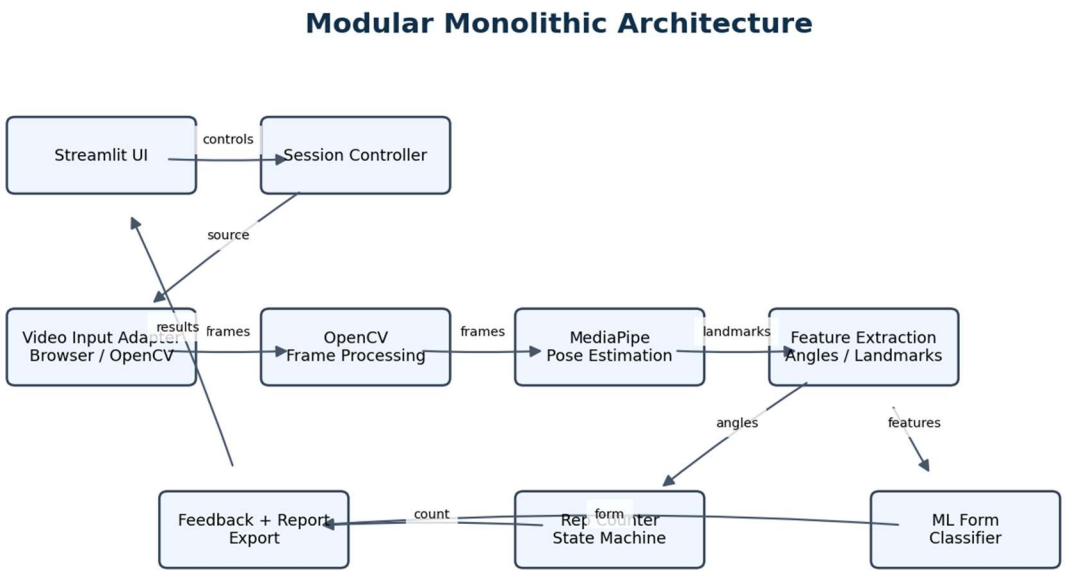


Figure 2. Proposed modular monolithic architecture.

9. Data Flow and System Behavior

The context diagram shows the User and camera/video source as external entities. The Level 1 DFD decomposes the application into capture, preprocessing, pose estimation, feature extraction, classification, repetition counting, feedback, presentation, and report export.

During a workout, the user selects an exercise and source; the application initializes the source; frames are processed; landmarks and features are extracted; the classifier and repetition counter update outputs; feedback is generated; and Streamlit refreshes the live view. This cycle repeats until the user stops the session.

On stop, the camera is released and a temporary session summary is finalized. The user may export the summary as a file. The MVP does not persist the summary in a database.

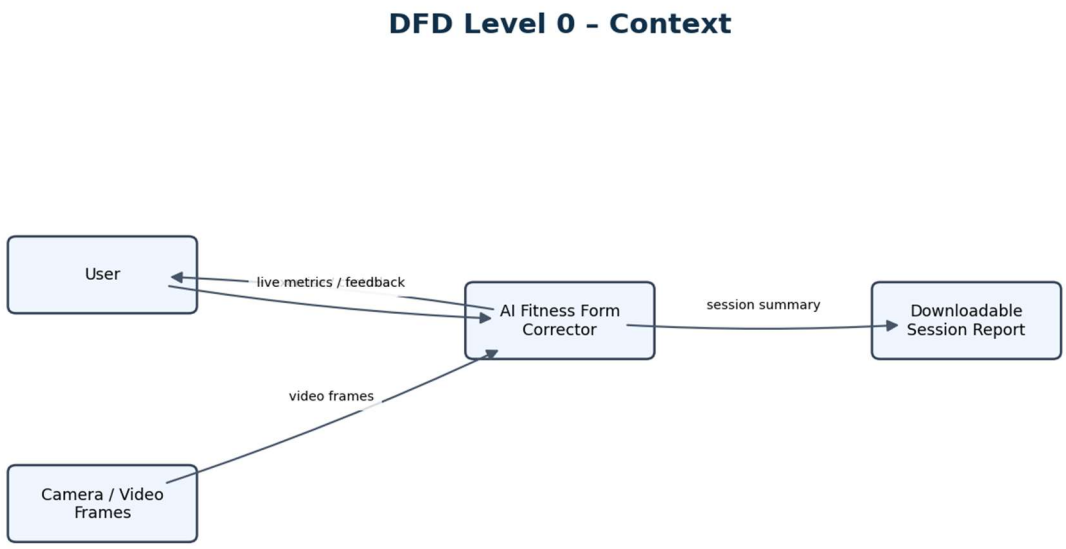


Figure 3. DFD Level 0 – context diagram.

10. DFD Level 1 – Detailed Processing

1.0 Video Capture obtains frames from the selected browser camera or OpenCV device. 2.0 Frame Preprocessing resizes and converts frames as required. 3.0 Pose Estimation detects body landmarks and visibility information. 4.0 Feature Extraction calculates exercise-specific angles and relative positions.

5.0 Form Classification applies the selected trained classifier. 6.0 Repetition Counting updates an exercise-specific state machine. 7.0 Feedback Generation combines form, count, and movement state. 8.0 Live UI and Report Export displays metrics and creates a downloadable summary after the session ends.

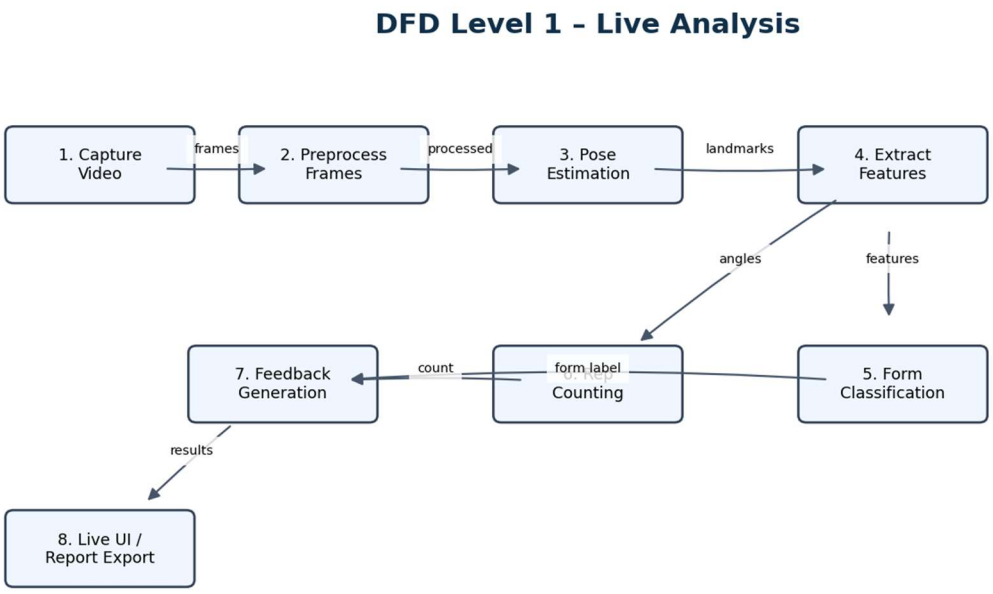


Figure 4. DFD Level 1 – detailed live-analysis pipeline.

11. Sequence Diagram – Live Workout

The sequence shows interaction between the user, Streamlit UI, video adapter, vision processing, AI analysis, and feedback engine. The adapter supplies frames; vision processing returns landmarks/features; AI analysis returns form and repetition outputs; feedback is displayed in the UI.

The cycle repeats while the session is active. If a frame is unavailable or pose quality is insufficient, the application shows a recoverable status and avoids misleading output. On stop, the source is released and the summary becomes available for export.

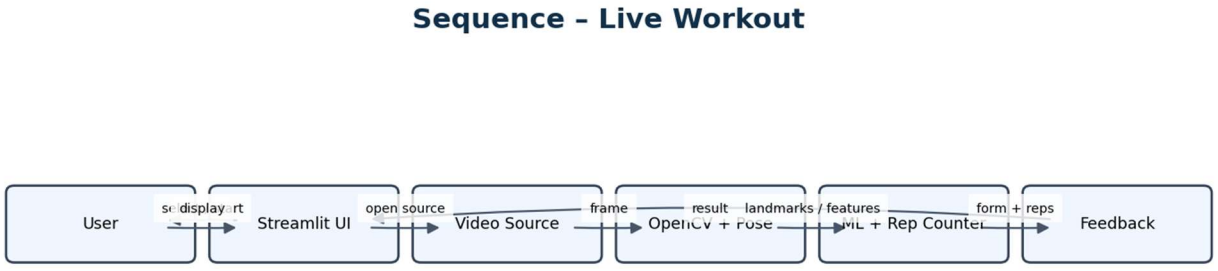


Figure 5. Sequence diagram for a live workout.

12. Activity Diagram – Workout Workflow

The user opens the application, selects an exercise and video source, and initializes the camera and model. For each frame, pose landmarks are detected and checked. If quality is inadequate, the system retries or requests repositioning without issuing unreliable feedback. If valid, features are calculated, form is evaluated, repetition count is updated, and the UI is refreshed. The loop ends when the user stops the session; the summary is finalized and can be exported.

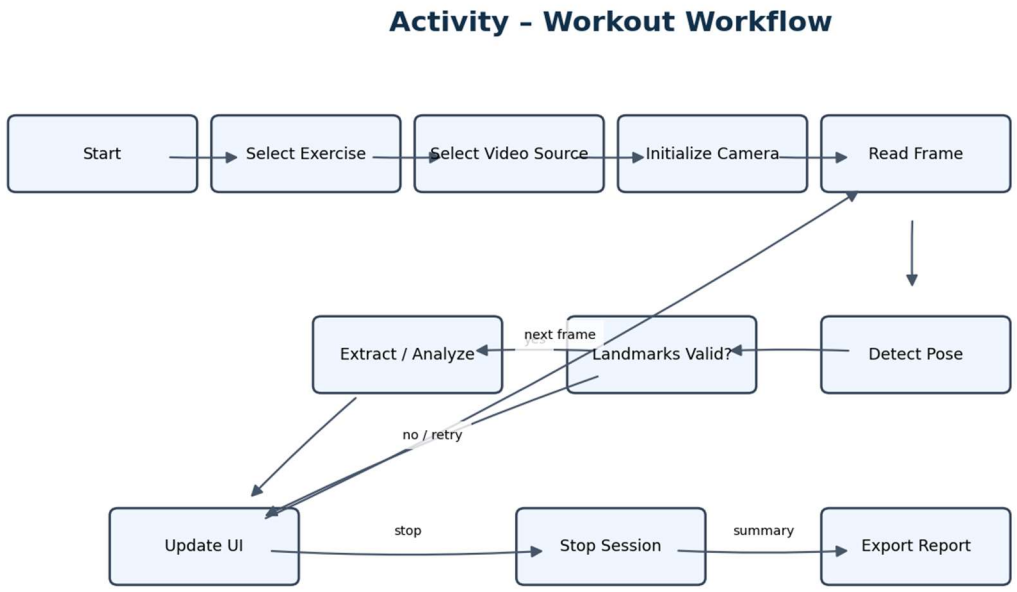


Figure 6. Activity diagram for workout workflow.

13. State Diagram – Repetition Counter

The repetition counter uses exercise-specific states: READY, DESCENDING, BOTTOM, ASCENDING, REP COMPLETED, and RESET/COOLDOWN. Transitions depend on validated angle thresholds, movement direction, and stability conditions.

A repetition is incremented only after the required sequence completes. Reset/cooldown logic helps prevent duplicate counts caused by jitter. Exact thresholds must be calibrated separately for each exercise. The diagram is conceptual and does not prescribe unvalidated numeric thresholds.

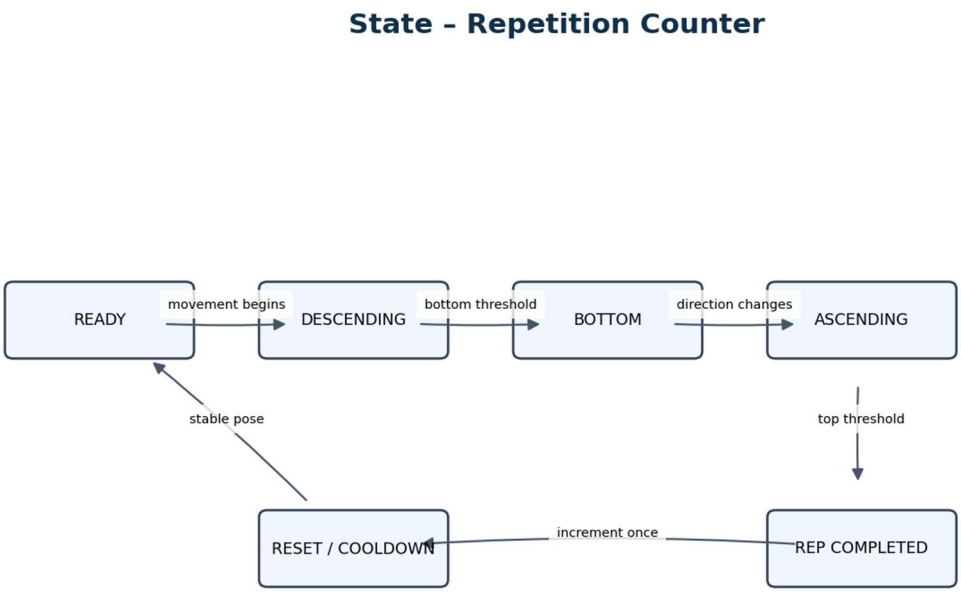


Figure 7. Conceptual repetition-counting state diagram.

14. Class Diagram – Logical Design

The logical class view identifies responsibilities: FitnessApp coordinates the entry point; WorkoutSession manages lifecycle and summary; VideoSource abstracts camera access; FrameProcessor prepares frames; PoseEstimator detects landmarks; FeatureExtractor calculates features; FormClassifier predicts form; RepCounter tracks movement phases; and ReportGenerator exports the summary.

These names represent logical responsibilities. The implementation may use Python classes, functions, or service modules as long as interfaces remain clear.

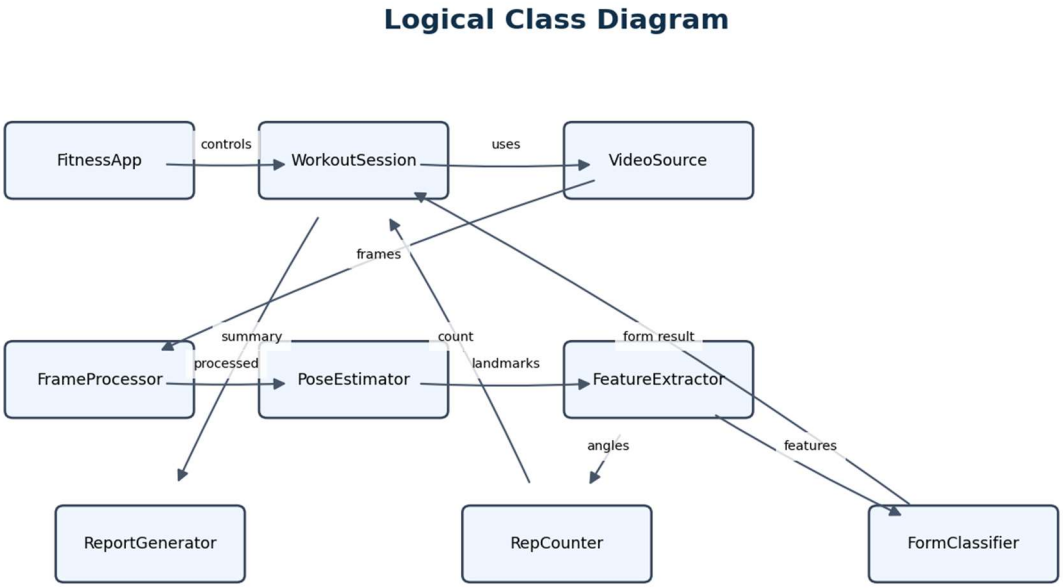


Figure 8. Logical class diagram.

15. UI/UX Design and Prototyping

The interface prioritizes a simple workout flow and clear live feedback. The Dashboard explains the project and supported exercises. Exercise Selection lets the user choose an exercise. Live Workout presents the camera view, optional pose overlay, repetition count, form status, and corrective feedback. Session Results summarizes the completed workout and offers a report download.

Screen	Main Elements	Primary Action
Dashboard	Project overview, instructions, supported exercise cards.	Open exercise selection.
Exercise Selection	Squat, Push-up, Biceps Curl and descriptions.	Choose exercise.
Live Workout	Camera view, pose overlay if available, count, form status, feedback, stop.	Monitor and stop workout.
Session Results	Exercise, duration, reps, form summary, feedback, export.	Download report.

16. UI/UX Guidelines

- Use a consistent hierarchy: exercise name, camera view, repetition count, form status, and feedback.
- Use a restrained palette; never communicate correctness by color alone.
- Use readable sans-serif typography and clear heading/body sizes.
- Keep feedback concise, actionable, and exercise-specific.
- Show camera permission, unavailable camera, low landmark confidence, and processing states.
- Provide clear Start, Stop, and Export Report actions.
- Explain camera processing and avoid unnecessary data collection.
- Support common laptop screen sizes and keyboard navigation where practical.

17. Component Diagram

The component view describes logical modules and dependencies. Streamlit UI calls the session controller; the controller selects a video input module; the vision module supplies landmarks and features to the classifier and repetition counter; outputs are combined by the feedback engine, which updates the UI and provides a summary to the report exporter.

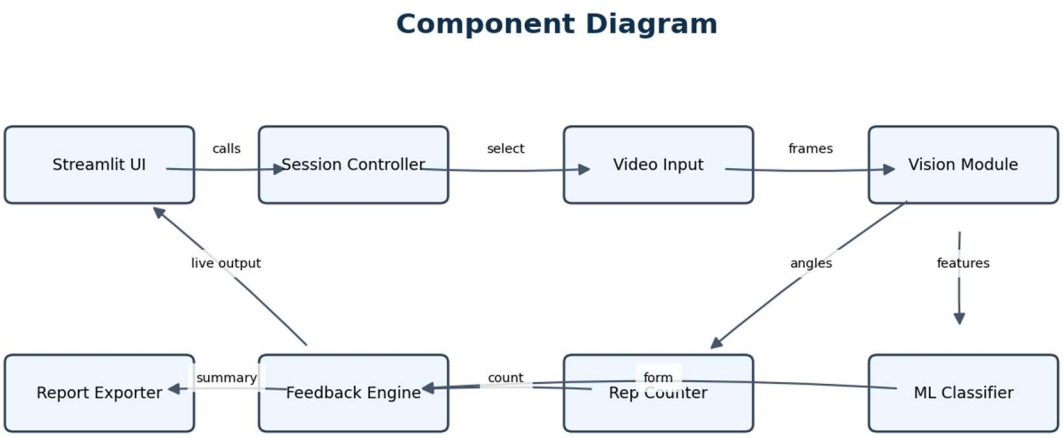


Figure 9. Component diagram.

18. Deployment Diagram and Strategy

The application is expected to run locally during development and may be packaged in Docker for Microsoft Azure. The hosting environment needs a compatible Python runtime and sufficient compute resources for pose estimation and ML inference.

Camera access is an important deployment constraint. Browser-based camera input is generally more suitable for remote hosting because the camera is attached to the user's device. OpenCV device-camera capture usually accesses a camera attached to the machine running the Python process; therefore, that option is primarily intended for local execution or a server with a directly attached camera. Validate this distinction before choosing the production deployment mode.

Proposed release pipeline: maintain code in GitHub; run automated tests; build a Docker image; configure environment variables and model artifacts; deploy to the selected Azure compute service; run smoke tests; and monitor health, latency, and errors. The exact Azure service should be selected after confirming camera requirements, resources, budget, and institutional constraints.

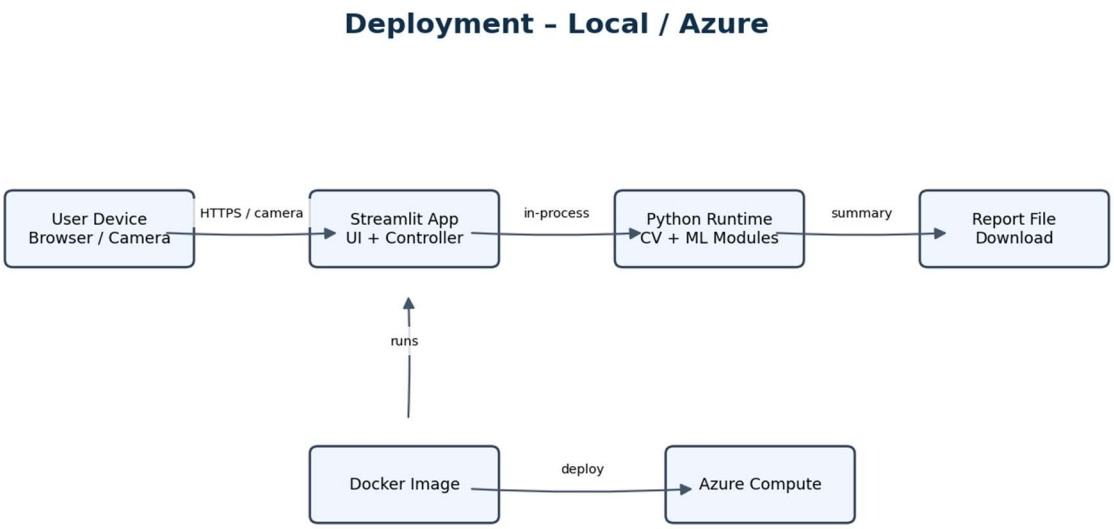


Figure 10. Conceptual deployment diagram.

19. Technology Stack

Layer	Technology	Role
User Interface	Streamlit	Dashboard, exercise selection, live metrics, report download.
Video Processing	OpenCV	Frame reading, preprocessing, image operations, local camera capture.
Pose Estimation	MediaPipe Pose	Body landmark detection.
Machine Learning	Scikit-learn / XGBoost	Candidate form classifiers; final model selected by validation.
Feature Engineering	Python / NumPy	Joint angles and feature preparation.
Report Export	Python reporting libraries	Generate downloadable session summary.
Version Control	Git / GitHub	Collaboration and version history.
Packaging	Docker	Reproducible application packaging.
Cloud	Microsoft Azure	Planned hosting environment.

20. API and Internal Interface Documentation

The MVP does not require a public REST API. Streamlit and processing modules can communicate through internal Python interfaces. The following contracts are logical proposals; exact names and signatures may be adjusted to the codebase.

Interface	Input	Output	Responsibility
VideoSource.open()	Source type/settings	Ready status or error	Initialize source.

Interface	Input	Output	Responsibility
VideoSource.read()	Active source	Frame or status	Acquire frame.
FrameProcessor.preprocess(frame)	Raw frame	Processed frame	Prepare input.
PoseEstimator.detect(frame)	Processed frame	Landmarks/confidence	Estimate pose.
FeatureExtractor.extract(landmarks)	Landmarks	Features and angles	Calculate model inputs.
FormClassifier.predict(features, exercise)	Feature vector/exercise	Label and confidence	Predict form.
RepCounter.update(angles, exercise)	Angles/exercise	Count and state	Update repetition logic.
FeedbackEngine.generate(result)	Form, confidence, count	Feedback object	Create guidance.
ReportGenerator.export(summary)	Session summary	Downloadable file	Export report.

21. Testing and Validation Plan

Testing must cover software behavior and model quality separately. A high classification score does not prove that live camera capture, repetition counting, or report export works correctly.

Level	Test	Expected Evidence
Unit	Feature extractor with known landmark coordinates.	Angles/features match expected values within tolerance.
Unit	Rep counter with a known angle sequence.	Exactly one completed repetition.
Unit	Rep counter with threshold jitter.	No duplicate/unstable counting beyond defined tolerance.
Unit	Report generator with sample summary.	Valid report with correct fields.
Integration	Camera frames through preprocessing and pose estimation.	Landmarks or clear low-confidence state.
Integration	Features through classifier and feedback engine.	Valid form result and feedback object.
Integration	Stop workout.	Camera released and summary finalized.
Integration	Browser and OpenCV input modes separately.	Each mode behaves as documented in intended environment.
Model	Held-out subject/video samples.	Accuracy, precision, recall, F1, confusion matrix.
Performance	Measure FPS and end-to-end latency.	Measured values recorded on target hardware.
UAT	Users complete each supported exercise workflow.	User can select, start, monitor, stop, export.

22. Model Evaluation and Acceptance Criteria

Use a leakage-aware split, preferably subject-wise where dataset organization permits it. Avoid distributing frames or clips from the same individual across training and test partitions if this would inflate performance.

Report accuracy, precision, recall, F1-score, and confusion matrix for form classification. Evaluate repetition counting against manually annotated counts using absolute error or mean absolute error. Measure FPS and end-to-end latency.

Numerical acceptance thresholds must be agreed with the supervisor/team after establishing a baseline. This document does not invent model accuracy, FPS, or latency achievements.

- All three exercise selections can be opened and analyzed.
- Both camera input paths are tested in their intended execution environments.
- Camera denial, disconnection, and low-confidence pose are handled.
- Rep counting is validated with annotated sequences for each exercise.
- Exported report contains correct exercise, duration, repetition count, and summary.
- Evaluation results are reproducible and include dataset split details and limitations.

23. Risks and Mitigation

Risk	Impact	Mitigation
Poor lighting / camera angle	Landmark detection unreliable.	Show setup guidance; validate camera position; handle low confidence.
Occlusion / partial body	Incorrect features and classification.	Define framing requirements; detect missing landmarks.
Dataset/live mismatch	Performance drop in real use.	Subject-wise validation; live-condition testing; document limits.
Rep-count jitter	Over-count or missed reps.	State transitions, hysteresis, cooldown, exercise-specific thresholds.
High latency	Delayed feedback.	Profile pipeline; resize frames; optimize inference.
Hosted camera constraints	OpenCV may not access remote user's camera.	Prefer browser camera input for hosted use.
Model artifact mismatch	Runtime errors or inconsistent output.	Version model artifacts and pin dependencies.
Overstated feedback	User may interpret output as medical advice.	Use cautious wording and state limitations.

24. Privacy, Safety and Limitations

The MVP does not require accounts or persistent workout history. Camera frames are processed for live analysis; raw video should not be saved persistently unless a separate requirement and consent process is introduced. The report should contain only information needed to summarize the workout.

The system is sensitive to camera placement, lighting, occlusion, visibility, and movement speed. Model outputs are estimates and may be wrong. The application must not claim to diagnose injuries, assess medical conditions, or guarantee safe exercise execution.

If privacy blur or anonymization is implemented, test that it does not remove landmarks needed for pose estimation. Face blur and body blur have different effects on the vision pipeline.

25. Future Enhancements

- LLM-based fitness assistant for natural-language questions and contextual explanations.
- User accounts and persistent workout history, subject to privacy and security design.
- Additional exercises and expanded pose/form models.

- Voice feedback and accessibility improvements.
- Mobile application or edge deployment.
- Temporal deep-learning models such as LSTM, GRU, or Transformer approaches if justified by data and evaluation.
- Personalized progress analytics and trainer-facing reports.

26. Conclusion

This design defines a practical MVP combining live video input, pose estimation, feature extraction, ML-based form classification, state-based repetition counting, feedback, and report export within a modular Python application using Streamlit.

The architecture avoids unnecessary infrastructure for the first release: no login, persistent database, or separate REST backend is required. It remains extensible so advanced features can be added after the core workflow is implemented and validated.